

# Improving the CMSA Algorithm with Online Deep Learning

Jaume Reixach<sup>a,\*</sup> and Christian Blum<sup>a</sup>

<sup>a</sup>Artificial Intelligence Research Institute (IIIA-CSIC), Campus of the UAB, Bellaterra, 08193, Spain  
ORCID (Jaume Reixach): <https://orcid.org/0009-0002-0305-9270>, ORCID (Christian Blum):  
<https://orcid.org/0000-0002-1736-3559>

**Abstract.** This work presents a new variant of construct, merge, solve, and adapt (CMSA), a hybrid metaheuristic for combinatorial optimization. This variant, denoted as DL-CMSA, implements an on-line deep learning mechanism, which allows it to learn how to construct solutions in its “construct” step. Compared to an existing variant of CMSA with a learning mechanism, the so-called RL-CMSA, the deep learning component of the presented variant allows for a higher degree of detail in the learning process, as the partial solution under construction is taken into account for deciding which solution component to add next. An experimental evaluation is performed on the classical Maximum Independent Set (MIS) problem. The results show how the proposed deep learning mechanism is beneficial, yielding statistically superior results.

## 1 Introduction

CMSA is a hybrid metaheuristic for combinatorial optimization introduced in [3]. It maintains, at all times, a sub-instance of the tackled problem instance, to which it applies an exact solver at each iteration. The goal is to modify the sub-instance throughout iterations so that it eventually contains a very good (if not optimal) solution that the exact solver can then easily find.

The application of CMSA requires defining a set of solution components  $C$  so that every solution can be expressed as a subset of  $C$ . The algorithm starts by initializing the subinstance  $C' \subseteq C$  as empty and, at each iteration, performs four steps that give the algorithm its name. The first step is the construct step, which consists of probabilistically generating several feasible solutions to the original problem instance. Next, the merge step adds the solution components of the constructed solutions to the subinstance  $C'$  and the solve step applies an exact solver restricted to  $C'$ . Finally, in the adapt step, the solution found by the exact solver ( $S_{\text{opt}}$ ) is used to reduce  $C'$  by removing those solution components that have not appeared in  $S_{\text{opt}}$  for a predefined number of iterations.

CMSA can be applied to any combinatorial optimization problem for which (1) solutions can be probabilistically generated, and (2) an exact solver is available. So far, CMSA has been successfully applied to a range of optimization problems, including [6, 8, 5].

Standard CMSA uses a probabilistic method for generating solutions that has to be tailored to the problem at hand, with fixed proba-

bilities throughout the search process. Recently, a variant called RL-CMSA was proposed [13]. This variant takes inspiration from reinforcement learning and implements a mechanism to learn to construct solutions online. RL-CMSA simplifies the standard algorithm and performs generally better than its standard counterpart. It works by learning about the quality of solution components through feedback from the exact solver. This learned knowledge is then used during solution construction. However, when constructing solutions, RL-CMSA does not consider the partial solution under construction to decide which solution component is added next. In this work, we enhance the learning mechanism of RL-CMSA using deep learning to consider partial solutions.

The rest of the paper is organized as follows. First, CMSA and RL-CMSA are introduced, after which the new proposed variant DL-CMSA is described. Finally, the three algorithms are experimentally evaluated on the Maximum Independent Set (MIS) problem, a classic combinatorial optimization problem.

### 1.1 Related work

This paper contributes to the expanding body of research on the use of machine learning in combinatorial optimization [2]. While some studies focus on end-to-end learning approaches, this work highlights the integration of machine learning into classical optimization methods, specifically a metaheuristic. For a comprehensive overview and taxonomy of machine learning applications in metaheuristics, we refer readers to the surveys [15, 14].

### 1.2 Standard CMSA and RL-CMSA

From now on, we assume to have a set  $C = \{c_1, c_2, \dots, c_m\}$  of solution components for the combinatorial optimization problem at hand that allows to express every solution  $S$  as a subset of  $C$ . For example, in the case of the Travelling Salesman Problem (TSP), the edges of the complete input graph might form set  $C$ . Algorithm 1 presents the structure of CMSA. Firstly, the best solution found so far ( $S_{\text{bsf}}$ ) is initialized to `NULL` and the subinstance  $C'$  as empty. Afterward, the main loop is entered, and the following four steps are iterated until a termination condition, usually a time limit, is reached.

- **Construct step (line 4):**  $n_a$  solutions are constructed using a probabilistic method designed for the problem at hand.
- **Merge step (lines 5 to 7):** the solution components present in the constructed solutions are added to subinstance  $C'$  in case they are

\* Corresponding Author. Email: [jaume.reixach@iiia.csic.es](mailto:jaume.reixach@iiia.csic.es).

The authors are supported by grant PID2022-136787NB-I00 funded by MCIN/AEI/10.13039/501100011033.

**Algorithm 1** Pseudo-code of standard CMSA

---

**Input 1:** Set  $C = \{c_1, \dots, c_n\}$  of solution components for the problem instance to be solved.  
**Input 2:** Parameter values  $n_a, t_{\text{solver}}, \text{age}_{\text{max}}$ .  
**Output:** Solution to the problem at hand.

```

1:  $S_{\text{bsf}} := \text{NULL}, C' := \emptyset$ 
2: while termination conditions not met do
3:   for  $j = 1, \dots, n_a$  do
4:      $S := \text{probabilistic\_solution\_construction}()$ 
5:     for all  $c_i \in S$  and  $c_i \notin C'$  do
6:        $\text{age}_{c_i} := 0$ 
7:        $C' := C' \cup \{c_i\}$ 
8:     end for
9:   end for
10:   $S_{\text{opt}} := \text{apply\_exact\_solver}(C', t_{\text{solver}})$ 
11:  if  $S_{\text{opt}}$  is better than  $S_{\text{bsf}}$  then  $S_{\text{bsf}} := S_{\text{opt}}$  end if
12:   $\text{adapt}(C', S_{\text{opt}}, \text{age}_{\text{max}})$ 
13: end while
14: return  $S_{\text{bsf}}$ 

```

---

not already there. Moreover, their so-called *age* value is initialized to zero.

- **Solve step (line 10):** the exact solver is applied restricted to  $C'$  within a time budget of  $t_{\text{solver}}$  CPU seconds.
- **Adapt step (line 12):** the age of the solution components in  $S_{\text{opt}}$  is set to zero, while the age of the ones in  $C' \setminus S_{\text{opt}}$  is increased by one. Finally, those components in  $C'$  whose age has reached  $\text{age}_{\text{max}}$  are erased from  $C'$ .

Note that  $n_a, t_{\text{solver}}$  and  $\text{age}_{\text{max}}$  are parameters of the algorithm.

Compared to CMSA, RL-CMSA features a learning mechanism based on keeping a quality measure for each solution component from  $C$ . These quality measures (called *q*-values) are used when constructing solutions by giving a higher probability to selecting solution components with a higher *q*-value. They are updated after every iteration depending on  $S_{\text{opt}}$ , as delivered by the exact solver. The *q*-values of solution components appearing in  $S_{\text{opt}}$  are increased, and the *q*-values of the ones in  $C'$ , but not appearing in  $S_{\text{opt}}$ , are decreased. Moreover, the algorithm implements a restart mechanism, which is triggered when the *q*-values become too unbalanced to avoid constructing solutions that are not diverse enough.

Algorithm 2 presents a pseudo-code for RL-CMSA. The difference between this algorithm and standard CMSA lies in the initialization of the *q*-values, the use of the *q*-values for constructing solutions, their update after the adapt step, and the optional restart mechanism. In more detail, *q*-values are all initialized to zero. Function  $\text{update\_q\_values}(\mathbf{q}, C', S_{\text{opt}})$  works by increasing the *q*-values of solution components in  $S_{\text{opt}}$  by one, respectively decreasing the ones of solution components in  $C' \setminus S_{\text{opt}}$  by one. Moreover, a solution  $S$  is constructed in  $\text{probabilistic\_solution\_construction}(\mathbf{q})$  by starting with  $S = \emptyset$  and iteratively adding a solution component until reaching a complete solution. Given the set  $C_{\text{feas}} \subseteq C$  of available solution components for extending a partial solution  $S$ , the next solution component added to  $S$  is selected as follows.

1. With probability  $dr$ , a random solution component from  $C_{\text{feas}}$  among those with the maximum *q*-value is selected.
2. Otherwise, the selection is done probabilistically by associating the following probability  $p_i$  to each  $c_i \in C_{\text{feas}}$ :

$$p_i = \frac{e^{\beta q_i}}{\sum_{c_k \in C_{\text{feas}}} e^{\beta q_k}}.$$

**Algorithm 2** Pseudo-code of RL-CMSA

---

**Input 1:** Set  $C = \{c_1, \dots, c_n\}$  of solution components for the problem instance to be solved.  
**Input 2:** Parameter values  $n_a, t_{\text{solver}}, \text{age}_{\text{max}}, cf_{\text{limit}}, b_{\text{reset}}$ .  
**Output:** Solution to the problem at hand.

```

1:  $S_{\text{bsf}} := \text{NULL}, C' := \emptyset$ 
2: for  $i = 1, \dots, n$  do  $q_i := 0$  end for
3: while termination conditions not met do
4:   for  $j = 1, \dots, n_a$  do
5:      $S := \text{probabilistic\_solution\_construction}(\mathbf{q})$ 
6:     for all  $c_i \in S$  and  $c_i \notin C'$  do
7:        $\text{age}_{c_i} := 0$ 
8:        $C' := C' \cup \{c_i\}$ 
9:     end for
10:   end for
11:   $S_{\text{opt}} := \text{apply\_exact\_solver}(C', t_{\text{solver}})$ 
12:  if  $S_{\text{opt}}$  is better than  $S_{\text{bsf}}$  then  $S_{\text{bsf}} := S_{\text{opt}}$  end if
13:   $\text{adapt}(C', S_{\text{opt}}, \text{age}_{\text{max}})$ 
14:   $\text{update\_q\_values}(\mathbf{q}, C', S_{\text{opt}})$ 
15:   $cf := \text{compute\_convergence\_factor}(\mathbf{q})$ 
16:  if  $cf > cf_{\text{limit}}$  then
17:    for  $i = 1, \dots, n$  do  $q_i := 0$  end for
18:    if  $b_{\text{reset}} = \text{true}$  then  $C' := \emptyset$  end if
19:  end if
20: end while
21: return  $S_{\text{bsf}}$ 

```

---

Hereby,  $\beta \geq 0$  and  $dr \in [0, 1]$  are parameters of the algorithm, which govern the exploration-exploitation balance with respect to the *q*-values.

Finally, the restart mechanism is triggered whenever the so-called convergence factor  $cf$  is greater than parameter  $cf_{\text{limit}} > 0$ . The computation of  $cf$  is done in  $\text{compute\_convergence\_factor}(\mathbf{q})$  as follows. The last constructed solution  $S$  is considered. For each solution component  $c_i \in S$ , the probability  $z_i$  of choosing  $c_i$  over the solution components not belonging to  $S$  is computed. The convergence factor is then returned as the minimum of these values. Note that, due to the solution component selection mechanism, the probability  $z_i$  can be simply computed as follows:

$$z_i = dr \cdot \chi_i + (1 - dr) \cdot \frac{e^{\beta q_i}}{\sum_{c_k \notin S} e^{\beta q_k} + e^{\beta q_i}}, \quad (1)$$

where  $\chi_i$  is defined as:

$$\chi_i := \begin{cases} \frac{1}{|\{c_k \in \hat{C}_i | q_k = q_i\}|} & \text{if } q_i = \max \{q_k \mid c_k \in \hat{C}_i\} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

and  $\hat{C}_i := (C \setminus S) \cup \{c_i\}$ . The restart process consists of re-setting the quality values back to zero and emptying subinstance  $C'$  depending on a boolean parameter  $b_{\text{reset}}$  (see lines 16 to 19 of Algorithm 2).

## 2 DL-CMSA

As mentioned before, RL-CSMA does not consider the partial solution under construction when deciding which solution component to choose next. In other words, the *q*-values are independent of the solution being constructed. However, the current partial solution at each solution construction step clearly influences the quality of available solution components at that moment. Therefore, it is a natural idea to extend RL-CMSA in that direction. The first idea that comes to mind

is to mirror the process used in RL-CSMA by maintaining separate  $q$ -values for each possible partial solution. However, this is clearly not feasible in practice due to the vast number of possible partial solutions.

The alternative we present in this work is to approximate these separate vectors of  $q$ -values using a neural network  $Q$ . To do this,  $Q$  takes a one-hot encoding of a partial solution as input and delivers one value as output for each solution component, representing the quality of using that solution component for extending the current partial solution. DL-CMSA assumes a specific order of the solution components to generate these one-hot encodings. Similarly to the  $q$ -values in RL-CMSA, the parameters of  $Q$  are updated at the end of every iteration of DL-CMSA, depending on the solution delivered by the exact solver. We propose a gradient descent step that updates the parameters of  $Q$  to increase the likelihood of the selections made by the exact solver.

A problem lies in the fact that the order in which the solver constructed its solution is generally unknown. To solve this issue solutions produced by the solver are always assumed to have been constructed in the order of the solution components as defined for the one-hot encoding. Given the ordered solution  $S_{\text{opt}} = (c_{k_1}, c_{k_2}, \dots, c_{k_m})$  provided by the solver, the parameters of  $Q$  are updated with the goal of increasing the output of  $Q$  for the corresponding list of (partial solution, solution component) pairs:

$$\{(\emptyset, c_{k_1}), (\{c_{k_1}\}, c_{k_2}), (\{c_{k_1}, c_{k_2}\}, c_{k_3}), \dots, (\{c_{k_1}, \dots, c_{k_{m-1}}\}, c_{k_m})\}. \quad (3)$$

This is achieved by setting a target for each partial solution as the output of  $Q$  for that solution, with the component corresponding to the associated solution component incremented by one (see line 16 of Algorithm 3). The loss is then computed (line 17), and a gradient descent step is performed using Adam [10] on the average loss (line 19). In this context, we use the notation  $Q_{\theta}(\phi(\hat{S}))$ , where  $\phi(\hat{S})$  is the one-hot encoding of the partial solution  $\hat{S}$ , and  $\theta$  denotes the parameters of the neural network being updated.

The neural network  $Q$  is used for constructing solutions in the construct step of DL-CMSA. Similarly to RL-CMSA, a solution construction starts with an empty solution to which solution components are iteratively added. The neural network  $Q$  replaces the  $q$ -values in this process. At each construction step, the next solution component is selected using epsilon-greedy selection in this case, as this method seemed to perform better for this algorithm. Given the set  $C_{\text{feas}} \subseteq C$  of available solution components for extending a partial solution  $S$ , the next solution component added to  $S$  is selected as follows.

1. With probability  $\varepsilon$ , a random solution component from  $C_{\text{feas}}$  is selected.
2. Otherwise, a random solution component between the ones from  $C_{\text{feas}}$  with a maximum output for  $Q$  is selected.

Notably,  $\varepsilon$  is initialized to 1 at the start of the algorithm and controlled by two parameters:  $\varepsilon_{\text{dec}} \in [0, 1)$  and  $\varepsilon_{\text{min}} \in [0, 1]$ , which control how much  $\varepsilon$  is decreased at each iteration and the minimum  $\varepsilon$  allowed respectively.

Algorithm 3 shows the general structure of DL-CMSA. As explained, the neural network  $Q$  replaces the vector  $q$  of RL-CMSA for constructing solutions, and the gradient descent step replaces the simple  $q$ -value update of RL-CMSA. Notably, DL-CMSA does not implement a restart mechanism as no issues were observed with convergence, which can be avoided by having a large enough  $\varepsilon_{\text{min}}$ .

---

### Algorithm 3 Pseudo-code of DL-CMSA

---

**Input 1:** Ordered set  $C = (c_1, \dots, c_n)$  of solution components for the problem instance to be solved.

**Input 2:** Parameter values  $n_a, t_{\text{solver}}, \text{age}_{\text{max}}, \varepsilon_{\text{dec}}, \varepsilon_{\text{min}}$ .

**Output:** Solution to the problem at hand.

```

1:  $S_{\text{bsf}} := \text{NULL}$ ,  $C' := \emptyset$ ,  $\varepsilon := 1$ 
2: Initialize the weights of neural network  $Q$  randomly
3: while termination conditions not met do
4:   for  $j = 1, \dots, n_a$  do
5:      $S := \text{probabilistic\_solution\_construction}(Q, \varepsilon)$ 
6:     for all  $c_i \in S$  and  $c_i \notin C'$  do
7:        $\text{age}_{c_i} := 0$ 
8:        $C' := C' \cup \{c_i\}$ 
9:     end for
10:   end for
11:    $S_{\text{opt}} := \text{apply\_exact\_solver}(C', t_{\text{solver}})$ 
12:   if  $S_{\text{opt}}$  is better than  $S_{\text{bsf}}$  then  $S_{\text{bsf}} := S_{\text{opt}}$  end if
13:    $\text{adapt}(C', S_{\text{opt}}, \text{age}_{\text{max}})$ 
14:   Order the solution components in  $S_{\text{opt}} := (c_{k_i})_{i=1}^m$ 
15:   for  $(\hat{S}, c_i) \in \{(\{c_{k_j}\}_{j=1}^i, c_{k_{j+1}})\}_{j=0}^{m-1}$  do
16:     Set target  $y^i := Q_{\theta}(\phi(\hat{S}))$ ,  $y_{c_i}^i := y_{c_i}^i + 1$ 
17:     Compute loss  $\mathcal{L}(\varphi) := (Q_{\varphi}(\phi(\hat{S})) - y^i)^2$ 
18:   end for
19:   Apply gradient descent step on average loss
20:   if  $\varepsilon > \varepsilon_{\text{min}}$  then  $\varepsilon := \varepsilon \cdot \varepsilon_{\text{dec}}$  end if
21: end while
22: return  $S_{\text{bsf}}$ 

```

---

## 3 Experimental Evaluation

In this section, we experimentally evaluate CMSA, RL-CMSA, and DL-CMSA on the Maximum Independent Set (MIS) problem [16]. The MIS problem is a well-known NP-hard combinatorial optimization problem in graph theory, which can be described as follows. Given an undirected graph  $G = (V, E)$  and  $v \in V$ , we denote by  $N(v) \subset V$  the set of neighbors of  $v$  and by  $N[v] := N(v) \cup \{v\}$  the so-called closed neighborhood of  $v$ . A set  $S \subset V$  is a feasible solution to the MIS problem if, for any pair  $v \neq u \in S$ , we have that  $(v, u) \notin E$ . In other words,  $S$  is a feasible solution if no two nodes from  $S$  are neighbors in  $G$ . The aim of the problem is to find a largest feasible solution.

Applying any of the three presented CMSA variants to a combinatorial optimization problem requires defining the set of solution components ( $C$ ) and choosing an exact solver. Additionally, standard CMSA specifically requires a method for probabilistically generating solutions. As for the set of solution components, we adopt a straightforward approach by defining  $C$  as the set of nodes of the input graph, that is:  $C = V$ . Concerning the exact solver, we use the commercial solver CPLEX with a standard ILP model for the MIS problem. This model employs a binary variable  $x_i$  for each node  $v_i \in V$  and is formulated as follows.

$$\max \sum_{v_i \in V} x_i \quad (4)$$

$$\begin{aligned} \text{subject to } & x_i + x_j \leq 1 & \forall e = (v_i, v_j) \in E \\ & x_i \in \{0, 1\} & \forall v_i \in V \end{aligned} \quad (5)$$

Notably, the solve step applies the exact solver restricted to the subinstance  $C'$ . In practice, this is done by adding the following constraints:  $x_i = 0 \forall v_i \notin C'$ .

**Algorithm 4** Method for probabilistically generating solutions of the standard CMSA for the MIS problem

**Input:** Parameter values  $dr_{CMSA}$ ,  $l_{size}$ .

**Output:** A solution to the MIS problem.

```

1:  $S := \emptyset$ 
2:  $\tilde{V} := V \setminus \bigcup_{v \in S} N[v]$ 
3: while  $\tilde{V} \neq \emptyset$  do
4:    $v := \text{choose\_from}(\tilde{V}, dr_{CMSA}, l_{size})$ 
5:    $S := S \cup \{v\}$ 
6:    $\tilde{V} := V \setminus \bigcup_{v_i \in S} N[v_i]$ 
7: end while
8: return  $S$ 

```

All three algorithms consider three Boolean parameters to control the behavior of CPLEX:  $cplex_{warmstart}$ ,  $cplex_{emphasis}$  and  $cplex_{abort}$ . The first parameter determines whether the best-so-far solution is provided to CPLEX as an initial solution. The second specifies whether CPLEX uses the highest heuristic emphasis value or defaults to the standard setting. The third controls whether a CPLEX execution is stopped as soon as a new best-so-far solution is found.

Finally, Algorithm 4 presents the method employed by standard CMSA for probabilistically generating solutions in its construct step. It consists of a simple probabilistic greedy method that starts with an empty solution to which it iteratively adds a node. The node is added from set  $\tilde{V} \subseteq V$ , which is the current set of nodes that do not have a single neighbor included in  $S$ . Function  $\text{choose\_from}(\tilde{V}, dr_{CMSA}, l_{size})$  selects a node as follows.

1. With a probability  $dr_{CMSA}$ , a random node of  $\tilde{V}$  between the ones that have the least amount of neighbors is selected.
2. Otherwise, a set  $L$  that contains the  $l_{size}$  nodes from  $\tilde{V}$  with less neighbors than the remaining nodes is built. A vertex is then selected from  $L$  uniformly at random.

Note that RL-CMSA and DL-CMSA do not require this method, as they construct solutions using the quality value vector  $q$  and the neural network  $Q$ , respectively. However, they both require a definition for the set of feasible solution components ( $C_{feas}$ ) at each construction step. And, in fact,  $C_{feas} := \tilde{V}$  is adopted for this purpose.

### 3.1 Problem Instances

The following three graph models are used for generating problem instances: Barabási-Albert (BA) [1], Watts-Strogatz (WS) [17], and Erdős-Rényi (ER) [7]. The first one generates graphs with a majority of low-degree nodes alongside a few nodes with significantly higher degrees. The second creates small-world networks, characterized by a short average shortest path length between nodes and a high level of local clustering. Lastly, the third is among the most well-known random graph models, generating graphs based on two parameters: the number of nodes and the probability of an edge existing between any node pair.

We generated 30 graphs for every combination of graph type, number of vertices  $|V| \in \{500, 1000, 1500, 2000\}$  and four different graph densities, controlled by parameters  $m$  (BA graphs),  $k$  (WS graphs), and  $p$  (ER graphs). The four corresponding density levels are achieved with  $p \in \{0.01, 0.04, 0.07, 0.1\}$  for all ER graph sizes,  $k, m \in \{3, 10, 17, 25\}$  for  $|V| = 500$ ,  $k, m \in \{5, 20, 35, 50\}$  for  $|V| = 1000$ ,  $k, m \in \{7, 30, 52, 75\}$  for  $|V| = 1500$  and  $k, m \in \{10, 40, 70, 100\}$  for  $|V| = 2000$ . From this point onward, the four density levels will be referred to as 1st, 2nd, 3rd, and 4th density levels, respectively. As a result, the benchmark set comprises 480 graphs

**Table 1.** Parameter values obtained after parameter tuning with *irace*. A dash (—) denotes that the algorithm does not use the respective parameter.

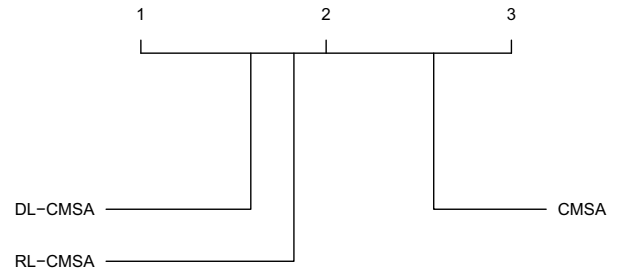
| Parameter           | Allowed Range                  | CMSA | RL-CMSA | DL-CMSA |
|---------------------|--------------------------------|------|---------|---------|
| $t_{solver}$        | $\{1, 2, \dots, 50\}$          | 20   | 9       | 9       |
| $n_a$               | $\{1, 2, \dots, 50\}$          | 6    | 23      | 7       |
| $age_{max}$         | $\{1, 2, \dots, 10\}$          | 3    | 1       | 3       |
| $cplex_{warmstart}$ | $\{0, 1\}$                     | 0    | 1       | 0       |
| $cplex_{emphasis}$  | $\{0, 1\}$                     | 1    | 1       | 1       |
| $cplex_{abort}$     | $\{0, 1\}$                     | 1    | 1       | 0       |
| $dr_{CMSA}$         | $\{0.0, 0.01, \dots, 0.99\}$   | 0.01 | —       | —       |
| $l_{size}$          | $\{3, 4, \dots, 100\}$         | 72   | —       | —       |
| $dr$                | $\{0.0, 0.01, \dots, 0.99\}$   | —    | 0.36    | —       |
| $\beta$             | $\{0.0, 0.01, \dots, 2.0\}$    | —    | 0.73    | —       |
| $b_{reset}$         | $\{0, 1\}$                     | —    | 1       | —       |
| $c_{f_{limit}}$     | $\{0.90, 0.91, \dots, 1.0\}$   | —    | 0.97    | —       |
| $n_{layers}$        | $\{1, 2, 3\}$                  | —    | —       | 1       |
| $n_{nodes}$         | $\{10, 11, \dots, 1000\}$      | —    | —       | 254     |
| $lr$                | $\{0.001, 0.002, \dots, 0.2\}$ | —    | —       | 0.199   |
| $\varepsilon_{dec}$ | $\{0.950, 0.951, \dots, 1.0\}$ | —    | —       | 0.959   |
| $\varepsilon_{min}$ | $\{0, 0.01, \dots, 0.10\}$     | —    | —       | 0.05    |

for each graph model, resulting in a total of 1440 instances. Additionally, it contains one extra instance for every graph type, density, and size for parameter-tuning purposes.

### 3.2 Parameter Tuning

The three CMSA variants are tuned using the *R* package *irace* [11], which implements iterated racing. Half of the tuning instances are used, with the instances corresponding to the 2nd and 3rd density levels excluded to speed up the procedure. This amounts to 24 tuning instances. The same budget of 3000 algorithm runs was allocated to the tuning process of the three algorithms. Table 1 presents the tuned parameters, their allowed value ranges, and the configurations generated by *irace*. Notably, in addition to the CMSA-related parameters, we tuned the number of hidden layers in the neural network  $Q$  of DL-CMSA ( $n_{layers}$ ), the number of hidden nodes per layer ( $n_{nodes}$ ), and the learning rate used by the Adam optimizer during the gradient descent step ( $lr$ ). The activation function was not tuned; instead, we opted for a Leaky ReLU [12] as the default choice.

The same time limits were applied to each tuning and evaluation run, consisting of 150, 300, 450, and 600 seconds for graphs/instances of 500, 1000, 1500, and 2000 nodes, respectively.



**Figure 1.** Critical Difference (CD) plot considering all problem instances.

### 3.3 Results

Table 2 provides the numerical results obtained. For each combination of graph size, graph type, and density level we present the average solution quality obtained by each algorithm, in addition to the average time spent for finding the corresponding solutions. Columns  $\bar{s}$  and  $\bar{t}_{best}$  contain these two values. As the instance set contains 30 graphs for each combination of size, type, and density, all numbers in the table are averages over 30 values.

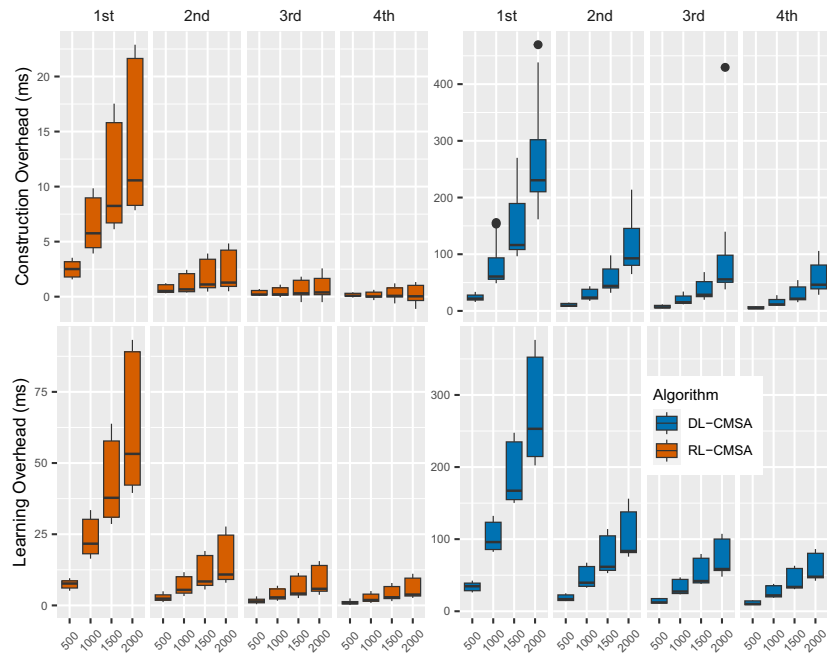
**Table 2.** Comparison of the three algorithms for the MIS problem instances.

| Size | Graph Type      | Density Level | CMSA          |                     | RL-CMSA       |                     | DL-CMSA       |                     |
|------|-----------------|---------------|---------------|---------------------|---------------|---------------------|---------------|---------------------|
|      |                 |               | $ s $         | $\bar{t}_{best}[s]$ | $ s $         | $\bar{t}_{best}[s]$ | $ s $         | $\bar{t}_{best}[s]$ |
| 500  | Barabási-Albert | 1st           | 242.47        | 5.15                | <b>242.53</b> | 0.46                | <b>242.53</b> | 1.19                |
|      |                 | 2nd           | 148.30        | 34.98               | <b>148.73</b> | 24.95               | 147.80        | 92.81               |
|      |                 | 3rd           | 111.67        | 39.73               | 112.50        | 39.29               | <b>112.53</b> | 52.83               |
|      |                 | 4th           | 88.80         | 51.08               | <b>89.43</b>  | 35.50               | 89.27         | 46.55               |
|      | Erdős Rényi     | 1st           | 217.90        | 21.77               | <b>217.93</b> | 21.74               | 217.73        | 62.96               |
|      |                 | 2nd           | 104.30        | 59.38               | 104.60        | 46.29               | <b>104.80</b> | 61.13               |
|      |                 | 3rd           | 71.67         | 58.07               | 71.97         | 68.78               | <b>72.30</b>  | 76.84               |
|      |                 | 4th           | 54.97         | 67.00               | 55.03         | 57.80               | <b>55.23</b>  | 81.87               |
|      | Watts-Strogatz  | 1st           | 174.90        | 18.67               | 175.03        | 27.46               | <b>175.20</b> | 38.90               |
|      |                 | 2nd           | 89.80         | 50.93               | <b>90.20</b>  | 49.78               | 89.90         | 68.66               |
|      |                 | 3rd           | 62.70         | 71.57               | <b>63.10</b>  | 55.73               | 62.97         | 86.87               |
|      |                 | 4th           | 46.93         | 59.72               | <b>47.57</b>  | 60.92               | 47.30         | 74.32               |
| 1000 | Barabási-Albert | 1st           | 403.77        | 82.40               | 406.17        | 53.88               | <b>406.23</b> | 86.80               |
|      |                 | 2nd           | 204.90        | 139.43              | 208.63        | 99.14               | <b>208.97</b> | 147.18              |
|      |                 | 3rd           | 147.27        | 138.37              | <b>149.17</b> | 142.64              | 148.20        | 125.04              |
|      |                 | 4th           | 116.20        | 120.38              | <b>116.93</b> | 139.75              | 113.40        | 134.49              |
|      | Erdős Rényi     | 1st           | 311.83        | 172.03              | 314.20        | 106.12              | <b>315.47</b> | 161.51              |
|      |                 | 2nd           | 131.07        | 197.69              | 131.00        | 139.05              | <b>131.40</b> | 236.06              |
|      |                 | 3rd           | 83.83         | 198.28              | 85.30         | 156.04              | <b>85.43</b>  | 235.78              |
|      |                 | 4th           | 63.17         | 176.71              | <b>64.00</b>  | 156.86              | 63.70         | 217.28              |
|      | Watts-Strogatz  | 1st           | 269.20        | 153.01              | 270.20        | 102.55              | <b>272.03</b> | 230.96              |
|      |                 | 2nd           | 109.73        | 144.60              | <b>111.87</b> | 157.73              | 110.17        | 233.01              |
|      |                 | 3rd           | 72.33         | 204.80              | 73.50         | 141.08              | <b>73.87</b>  | 223.63              |
|      |                 | 4th           | 54.70         | 186.87              | <b>55.30</b>  | 128.91              | 55.03         | 193.99              |
| 1500 | Barabási-Albert | 1st           | 522.93        | 219.24              | 529.23        | 138.89              | <b>530.43</b> | 250.63              |
|      |                 | 2nd           | 243.40        | 269.95              | 247.50        | 100.17              | <b>250.17</b> | 256.45              |
|      |                 | 3rd           | 172.37        | 213.55              | 173.17        | 148.83              | <b>174.70</b> | 279.92              |
|      |                 | 4th           | <b>133.00</b> | 214.62              | 132.63        | 155.81              | 132.80        | 238.52              |
|      | Erdős Rényi     | 1st           | 371.17        | 253.86              | 376.03        | 223.51              | <b>377.87</b> | 295.45              |
|      |                 | 2nd           | 142.67        | 293.73              | 142.53        | 176.77              | <b>144.80</b> | 345.35              |
|      |                 | 3rd           | 87.33         | 225.45              | 91.33         | 170.54              | <b>92.47</b>  | 322.01              |
|      |                 | 4th           | 64.23         | 227.44              | <b>68.40</b>  | 203.67              | 68.20         | 308.46              |
|      | Watts-Strogatz  | 1st           | 331.40        | 312.74              | 332.93        | 248.16              | <b>334.33</b> | 394.90              |
|      |                 | 2nd           | 119.57        | 259.25              | 123.10        | 233.47              | <b>123.67</b> | 351.93              |
|      |                 | 3rd           | 76.10         | 279.75              | 80.23         | 220.24              | <b>80.30</b>  | 319.77              |
|      |                 | 4th           | 56.83         | 305.76              | 59.50         | 165.15              | <b>59.60</b>  | 285.76              |
| 2000 | Barabási-Albert | 1st           | 591.23        | 399.37              | 604.13        | 220.43              | <b>606.43</b> | 444.02              |
|      |                 | 2nd           | 269.63        | 373.66              | 275.10        | 246.50              | <b>278.50</b> | 424.62              |
|      |                 | 3rd           | 188.20        | 325.21              | <b>189.03</b> | 245.82              | 181.43        | 318.55              |
|      |                 | 4th           | 145.47        | 331.82              | 143.50        | 208.51              | <b>145.73</b> | 387.46              |
|      | Erdős Rényi     | 1st           | 413.97        | 409.75              | 419.07        | 279.78              | <b>422.33</b> | 449.55              |
|      |                 | 2nd           | 147.17        | 291.38              | 151.30        | 274.14              | <b>151.80</b> | 452.41              |
|      |                 | 3rd           | 89.17         | 305.93              | <b>95.63</b>  | 274.19              | 94.57         | 405.05              |
|      |                 | 4th           | 64.50         | 250.79              | 70.10         | 251.55              | <b>70.50</b>  | 380.90              |
|      | Watts-Strogatz  | 1st           | 351.67        | 415.24              | 353.23        | 368.81              | <b>359.17</b> | 502.23              |
|      |                 | 2nd           | 123.67        | 297.47              | 129.60        | 271.44              | <b>131.90</b> | 487.21              |
|      |                 | 3rd           | 77.03         | 279.67              | <b>84.63</b>  | 314.65              | 84.10         | 469.82              |
|      |                 | 4th           | 56.90         | 285.13              | <b>62.97</b>  | 352.36              | 62.33         | 414.77              |

The obtained results demonstrate that DL-CMSA is the strongest performer, as it obtains the best average result much more frequently than the other two competitors. This is confirmed by Figure 1, which shows a critical difference (CD) plot concerning the obtained results. This plot indicates the average rank of each algorithm on the x-axis, with respect to the complete benchmark set. It was generated using the *R* package `scamp` [4]. Statistical significance was assessed using the Friedman rank-sum test, with Finner's procedure [9] as the post-hoc method for pairwise comparisons, applying a significance level of 0.05. As there is no bold horizontal line connecting the whiskers of two or more of the tested algorithms, we can say that the three algorithms perform differently with statistical significance. CD plots for each graph size (not shown due to space restrictions) reveal that differences between the algorithms are the most pronounced for the larger instances of 1500 and 2000 nodes. This is not surprising because the exact solver handles most of the workload for smaller

instances, whereas an intelligent management of the subinstance  $C'$  becomes more crucial for larger graphs. Notably, the differences between the algorithms were found to be statistically significant for instances with 2000 and 1500 nodes, while the differences between DL-CMSA and RL-CMSA were not statistically significant for the smaller instances of 1000 and 500 nodes.

Regarding computational time, RL-CMSA is the fastest algorithm to find its best solutions, followed by the standard CMSA. DL-CMSA, however, takes the longest to find its solutions, making a better use of the given time limits. This goes along with DL-CMSA suffering from the greatest learning overhead due to the network forward passes during solution construction and due to its update at the end of each iteration. Figure 2 presents the time overhead of the learning variants relative to the standard CMSA during solution construction (top) and the learning step (bottom). Both plots exhibit similar trends: DL-CMSA incurs significantly higher overhead in both



**Figure 2.** Time overhead for (top) the construction of a solution, and (bottom) the update of the learning procedure.

phases. This is expected, as DL-CMSA constructs solutions through iterative forward passes of a neural network and performs a gradient descent update for every solution component included in the CPLEX solution during its learning step. In contrast, RL-CMSA constructs solutions by referencing the precomputed  $q$ -values, and its learning step involves one update for each  $q$ -values associated with the current sub-instance.

The plots also show that overheads increase with graph size and decrease with graph density. This is because larger graphs and sparser instances result in more solution component selections during construction, as well as a greater number of components in the CPLEX-produced solutions. In the MIS problem specifically, the number of selected components corresponds to the solution size, which—due to the problem’s definition—increases with graph size and decreases with density. Additionally, for DL-CMSA, the input and output sizes of the neural network are equal to the graph size, further amplifying the overhead as graph size grows.

This growth suggests that for significantly larger or sparser instances, the performance of RL-CMSA and DL-CMSA may be adversely affected due to the excessive time required for their respective learning mechanisms. More generally, their performance might be affected for instances of combinatorial optimization problems that have solutions formed by a large number of solution components.

Finally, Figure 3 illustrates the quality of solutions constructed by the solution construction mechanisms of the three CMSA variants over time for four selected instances of each graph type. The chosen instances are some of those for which DL-CMSA outperformed RL-CMSA, aiming to better understand the factors behind DL-CMSA’s superior performance on this problem. In the presented runs, DL-CMSA achieved solutions with sizes approximately 2.12% to 5.88% larger than those found by RL-CMSA. Note that the quality of the solutions constructed in CMSA remains stable throughout the execution, as this algorithm does not incorporate any learning mechanism.<sup>1</sup>

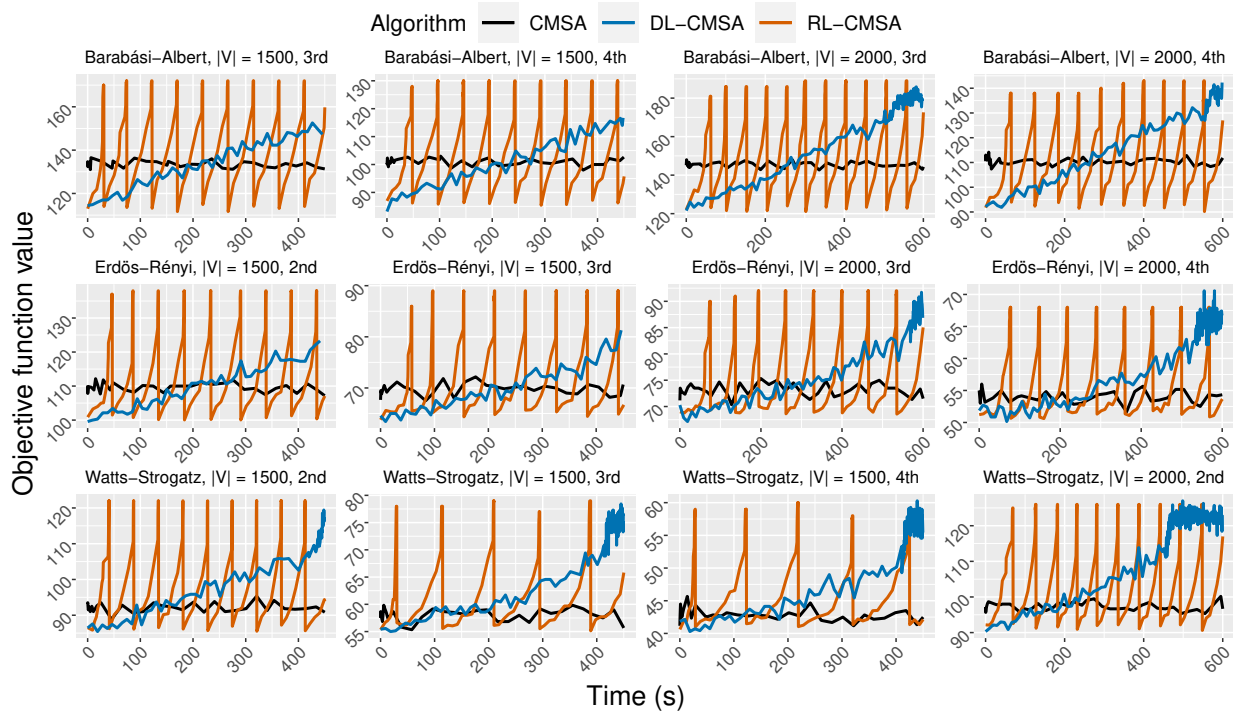
<sup>1</sup> The best-found solution by CMSA still improves over time, of course, due to the exact solver finding better and better solutions in the sub-instance.

In contrast, RL-CMSA exhibits a rapid learning behavior with multiple restarts during its execution. DL-CMSA, on the other hand, exhibits slower learning. Interestingly, while DL-CMSA generally does not learn to construct better solutions than RL-CMSA, it still outperforms the latter in terms of the best solution found at the end of a run. This likely indicates that DL-CMSA generates solutions with a more effective balance between quality and diversity, which is better for building higher-quality subinstances  $C'$ , rather than producing higher-quality solutions that are more similar among each other. Another observation is that DL-CMSA appears to learn more effectively for the instances of 2000 nodes, likely due to the longer execution times allowed. It would be interesting to compare the algorithms under extended time limits, as DL-CMSA might then learn to construct better solutions than RL-CMSA more consistently. Importantly, the improved performance of DL-CMSA does not stem from differences in the selection mechanisms. RL-CMSA was re-tuned and evaluated using a greedy epsilon-selection strategy, and the performance gap persisted.

## 4 Conclusions

This work introduced a new variant of the CMSA algorithm that incorporates an online deep learning mechanism. The algorithm, referred to as DL-CMSA, maintains a neural network  $Q$ , which takes one-hot encoded partial solutions as input and delivers a value for each solution component as output, indicating the quality of selecting the respective components to extend the given partial solution. The neural network  $Q$  is used for constructing solutions and is updated iteratively with feedback from the exact solver: its parameters are adjusted to make the selections performed by the exact solver more likely in future iterations.

We evaluated DL-CMSA on the Maximum Independent Set (MIS) problem, comparing it to standard CMSA and RL-CMSA, a variant from the literature that incorporates a simpler learning mechanism inspired by reinforcement learning. The results demonstrate the sta-



**Figure 3.** Quality of the solutions generated by the different construction mechanisms over time for a set of selected instances.

tistically significant superiority of DL-CMSA on larger instances, for which it generally achieves better outcomes. Additionally, we analyzed the overhead introduced by the learning mechanisms in both RL-CMSA and DL-CMSA, observing that this overhead increases with graph size and decreases with graph density. This indicates that for sufficiently large or sparse instances, the performance of these learning-based approaches may be impacted by an excessive computational overhead. Furthermore, we examined the quality of solutions constructed by the respective algorithms' solution construction mechanism over time, highlighting the distinct learning behaviors of RL-CMSA and DL-CMSA. Interestingly, while DL-CMSA outperformed the other methods overall, it did not consistently learn to construct better solutions than RL-CMSA, suggesting that factors such as solution diversity may play a key role in its success.

As for future work, larger instances of the same problem could be considered to determine the point at which the learning overhead becomes prohibitive. Additionally, DL-CMSA should be evaluated on a wider range of problems to assess whether other optimization problems also benefit from taking into account the partial solution during solution construction.

## References

- [1] A.-L. Barabási and R. Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999.
- [2] Y. Bengio, A. Lodi, and A. Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [3] C. Blum, P. Pinacho, M. López-Ibáñez, and J. A. Lozano. Construct, merge, solve & adapt a new general algorithm for combinatorial optimization. *Computers & Operations Research*, 68:75–88, 2016.
- [4] B. Calvo and G. Santafé Rodrigo. scmamp: Statistical comparison of multiple algorithms in multiple problems. *The R Journal*, Vol. 8/1, Aug. 2016, 2016.
- [5] M. Djukanović, A. Kartelj, and C. Blum. Self-adaptive CMSA for solving the multidimensional multi-way number partitioning problem. *Expert Systems with Applications*, page 120762, 2023.
- [6] N. Dupin and E.-G. Talbi. Matheuristics to optimize refueling and maintenance planning of nuclear power plants. *Journal of Heuristics*, 27(1-2):63–105, 2021.
- [7] P. Erdős and A. Rényi. On random graphs I. *Publ. math. debrecen*, 6(290-297):18, 1959.
- [8] J. Ferrer, F. Chicano, and J. A. Ortega-Toro. CMSA algorithm for solving the prioritized pairwise test data generation problem in software product lines. *Journal of Heuristics*, 27:229–249, 2021.
- [9] S. García, A. Fernández, J. Luengo, and F. Herrera. Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power. *Information sciences*, 180(10):2044–2064, 2010.
- [10] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [11] M. López-Ibáñez, J. Dubois-Lacoste, L. P. Cáceres, M. Birattari, and T. Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.
- [12] A. L. Maas, A. Y. Hannun, A. Y. Ng, et al. Rectifier nonlinearities improve neural network acoustic models. In *Proc. icml*, volume 30, page 3. Atlanta, GA, 2013.
- [13] J. Reixach and C. Blum. How to improve “Construct, Merge, Solve and Adapt”? use reinforcement learning! *Annals of Operations Research*, 2024. In press.
- [14] S. Szénási and G. Légrádi. Machine learning aided metaheuristics: A comprehensive review of hybrid local search methods. *Expert Systems with Applications*, 258:125192, 2024.
- [15] E.-G. Talbi. Machine learning into metaheuristics: A survey and taxonomy. *ACM Computing Surveys (CSUR)*, 54(6):1–32, 2021.
- [16] R. E. Tarjan and A. E. Trojanowski. Finding a maximum independent set. *SIAM Journal on Computing*, 6(3):537–546, 1977.
- [17] D. J. Watts and S. H. Strogatz. Collective dynamics of ‘small-world’ networks. *Nature*, 393(6684):440–442, 1998.